Git 学习笔记 + 极简速查表

> 仓库：JavaLearning｜主分支：main｜用途：Java 后端 + AI 工程学习打卡

一、环境初始化（整台电脑只做一次）

首次配置身份

```
# 设置用户名（和 GitHub 用户名一致）
git config --global user.name "你的GitHub用户名"

# 设置邮箱（GitHub 绑定的邮箱）
git config --global user.email "你的GitHub邮箱"

# 查看配置是否生效
git config --list
```

二、核心命令总表（14 个）

命令	作用说明	什么时候用
`git init`	在当前文件夹初始化 Git 本地仓库，生成隐藏的 `.git` 文件夹	新仓库，只做一次
`git clone 远程仓库地址`	把 GitHub 上的远程仓库完整下载到本地	换电脑、拿到别人的仓库
`git status`	查看仓库当前状态：哪些文件被修改、是否已加入暂存区	每次操作前先看
`git add 文件名`	把指定文件加入暂存区；`git add .` 表示当前目录全部改动	commit 之前
`git commit -m "备注"`	把暂存区内容提交到本地仓库，`-m` 后面写清楚本次改了什么	一个完整改动做完
`git log`	查看所有提交历史，拿到 commit-id（回滚要用）	回滚前、查改了什么

`git branch`	分支管理：无参数＝列出本地分支；`git branch 分支名` ＝新建分支	开新功能时
`git switch 分支名`	切换到指定分支（新版推荐，替代老的 checkout）	切换工作线
`git remote add origin 远程地址`	本地仓库绑定远程 GitHub 仓库，origin 是远程仓库默认别名	新仓库，只做一次
`git push origin 分支名`	把本地分支的提交推送到远程仓库	上传到 GitHub
`git pull origin 分支名`	拉取远程仓库最新代码并合并到本地	多人协作 / 换电脑后同步
`git merge 分支名`	把目标分支的代码合并进当前所在分支	功能做完合回主干
`git reset --hard commit-id`	版本回滚，强制回到指定提交版本；未提交的修改会丢失	只在练习仓库用
`git stash`	临时储藏写了一半、还不想提交的代码，切分支后再恢复	临时切分支救急

三、日常上传三板斧（最常用，直接复制）

日常一条龙

```
git add . && git commit -m "备注写这里" && git push
```

示例：

示例

```
git add . && git commit -m "更新 git 操作笔记" && git push
```

💡 **三步各干什么**：`git add .` 只是放进暂存区，不等于上传；`git commit` 才存成本地版本（断网也能存）；`git push` 才是真正推到 GitHub。

`&&` 表示前面成功才执行后面，中途报错会自动停下，比 `;` 安全。

只上传 Notes 文件夹

适合现在的目录结构

```
git add Notes/ && git commit -m "更新笔记" && git push
```

只做一次！初始化相关（不要再重复敲）

已执行过就跳过

```
git init
git remote add origin https://github.com/sngyijian-hub/JavaLearning.git
git branch -M main
```

关机重启后，第二天要上传怎么做

不用重新 `init`，也不用重新绑定远程，只做两件事：

1. 进到 `JavaLearning` 仓库根目录
2. 敲日常三板斧

开机后的固定流程

```
git status
git add . && git commit -m "本次修改备注" && git push
```

注：`-u` 只有第一次推送需要（`git push -u origin main`），之后直接 `git push`。

四、标准实操流程：从零到第一次推送

新建仓库完整链路

```
# 1. 创建本地文件夹并进入
mkdir git-demo
cd git-demo

# 2. 初始化本地 git 仓库
git init

# 3. 绑定远程仓库（替换成你自己的仓库地址）
git remote add origin https://github.com/xxx/git-demo.git

# 4. 创建/修改文件
echo "第一次测试git" > test.txt

# 5. 查看状态
git status
```

```
16
17  # 6. 添加到暂存区
18  git add .
19
20  # 7. 提交到本地仓库
21  git commit -m "第一次提交：新增test.txt"
22
23  # 8. 推送到远程 main 分支
24  git push origin main
```

五、分支练习流程

分支新建、切换、推送、合并

```
1   # 新建 dev 分支
2   git branch dev
3   # 切换到 dev 分支
4   git switch dev
5
6   # 在 dev 分支修改文件
7   echo "dev分支改动" >> test.txt
8   git add .
9   git commit -m "dev分支修改文件"
10  # 推送 dev 分支到远程
11  git push origin dev
12
13  # 切回主分支
14  git switch main
15  # 将 dev 分支合并到 main
16  git merge dev
```

六、版本回滚练习（D1 验收重点）

reset --hard 回滚

```
1   # 查看提交记录，复制想回退版本的 commit-id
2   git log
3
4   # 回滚到指定版本
5   git reset --hard 粘贴复制的commit-id
6
7   # 推送到远程（仅练习仓库使用！正式项目禁止随便 --force）
8   git push origin main --force
```

💡 `git reset --hard` 会直接丢掉所有未提交的修改，只在你自己练习的仓库里用；公共分支不要随便强推。

七、排错小命令

命令	用途
`git remote -v`	查看绑定的 GitHub 地址对不对
`git diff`	看文件具体改了什么内容
`git reset HEAD .`	取消 add，把文件移出暂存区（改动不会删）
`git config --list`	检查用户名、邮箱配没配对
`ls` / `pwd`	确认当前目录里有什么、自己在哪个目录

八、高频踩坑记录

1. `fatal: not a git repository`：当前目录没有 `.git`，要么 `git init`，要么进到正确的仓库目录。
2. `pathspec 'README.md' did not match any files`：文件不存在或名字不对。先用 `ls` 看真实文件名，懒人直接 `git add .`。
3. HTTPS 推送要密码：GitHub 不支持账号密码，用个人访问令牌 PAT，或配置 SSH 密钥。
4. 代码冲突：`<<<<<<< HEAD` 标记处就是冲突位置，手动改完再 add、commit。
5. `git reset --hard` 风险：丢掉所有未提交修改，只在自己练习仓库使用。
6. Git Bash 里 `Ctrl+V` 无效：粘贴用鼠标右键，或 `Shift + Insert`。

九、重点规则

1. 顺序：**add → commit → push**，三步缺一不可。
2. `&&`：前面成功才执行后面，出错自动停下，安全。
3. 关机重启后，不用重新 init、不用重新绑定远程，进到仓库目录直接用上传那一行。
4. 不要删仓库里隐藏的 `.git` 文件夹，删掉版本记录就没了。
5. 想一次敲多行：`git add . && git commit -m "备注" && git push` 一行回车即可跑完。

十、提交备注规范

简短写清楚做了啥，方便日后回看：

- `新增高数笔记`
- `修正 git 操作步骤`
- `补充 python 基础代码`

十一、D1 验收清单

- [ ] 配置 git 用户名、邮箱
- [ ] GitHub 新建仓库并绑定 origin
- [ ] 本地完成 init → add → commit → push
- [ ] 新建分支、切换分支、推送分支、合并分支
- [ ] git log 查看记录，执行一次版本回滚

（注：部分内容由豆包 AI 生成）